Project Planning and Requirements Proposal

Goal and Objectives:

The objective of our project is to design and develop a modern open-source **Ruby gem** for the **LTI v1.3 specification** to address the current lack of a maintained and updated LTI based gem (https://github.com/instructure/ims-lti).

To meet our goal we plan to implement the following objectives:

- Implement LTI v1.3 Core launch. Enables Ruby LMS applications to securely validate and launch external LMS tools Using OpenId Connect and JWT authentication. https://www.imsglobal.org/spec/lti/v1p3/#lti-launch
- Implement Assignment and Grade Services (AGS) for grading. Allows tools to report and manage grades back to their respective LMS while being compliant with the LTI standards. https://www.imsglobal.org/spec/lti-ags/v2p0/
- Implement Names and Roles Provisioning Services (NRPS) for roster management. Supports tools to retrieve and manage the information of course rosters and user roles. https://www.imsglobal.org/spec/lti-nrps/v2p0
- Provide clear and detailed documentation, use cases, and automated test cases to ensure future maintainability and correctness of the Ruby Gem.

Use Case & User Stories:

1. As a developer who will be using this gem, I want the library to have good coverage of the LTI 1.3 protocol, including at least the following core features:
    a. Secure Launch Tool
    b. Automated grading
    c. List of students and their grades for a marking report
2. As a developer who will be maintaining or contribute to this gem, I want the library in a generic ruby gem format following standard practices so I am familiar with the layout of the library allowing me to easily navigate it to verify correctness/behaviour and so that it's easily extensible by forking it and contributing to open source.
    a. Unit tests (rspec)
    b. README (what the gem does, how to use, the interface)
    c. Linting (Rubocop)
    d. In-line documentation (rdoc)
3. As a developer who will be using this gem, I want the library to have comprehensive validation, logging, and error checking so that my application can be robust and I can easily debug. This is important when integrating the library with my production applications.

Milestones & Timeline

Weeks 1 to 4 (Jan 4 to Jan 29)

- Meeting team
- Jan 29: Project Planning & Requirement Proposal

Week 5 to 6(Jan 30 to Feb 14)

- Setup: Github repo, workflow, coding standards(Rubocop, rspec)
- Learning and getting familiar with Ruby and the work environment
- Ruby Gem format
- Implement Core launch with testing + documentation (no dependency)
- 12th: Ethical Analysis and Responsible Design Decisions Document

Week 7 (Feb 15 to Feb 21)

- Based on previous weeks and launch implementation workflow, decide if we want to continue working through one user story at a time or work in parallel.
- Implement Grading with testing + documentation (Dependency: Core Launch)

Week 8 (Feb 22 to Feb 28)

- Implement Grading with testing + documentation
- Implement NRPS for roster management with Testing + documentation (no dependency)

Week 9 (March 1 to March 7)

- Implement NRPS for roster management with Testing + documentation
- Deep Linking Specification (Optional, if time permitting)*

Week 10 (March 8 to March 14)

- Finalize testing + documentation
- Address any final issues/feedback
- prepare for the presentation

Week 11 (March 15 to March 21)

- **Prepare for the presentation**
- 17th: Final Presentation Draft Due
- 19th: Final Presentation

Week 12 (March 29 to April 6)

- April 2nd: Final Software
- April 6th: White Paper

Roles

Glyn - LTI Core Launch Lead

- Responsible for the development of the LTI 1.3 Core launch feature, through the use of OpenID Connect, JWT validation, and claim parsing for tool launch authentications.

Dragon - AGS Grading Lead

- Leading the implementation of grading functionality, including line item management and score submissions through AGS APIs. Ensures correct interaction and reliable handling of any requests related to grades.

Ennes - NRPS Roster Management Lead

- Manages implementation of the roster management functionality, which includes membership requests and parsing by roles.

Ruili (Mary) - Documentation Lead

- Oversees the quality and clarity of documentation seen by user developers. This includes READMEs that outline what the gem does, how to use it, most common use cases, etc to API documentation and verifying the quality of the team's inline documentation.

Jessie - Testing Lead

- Designs and enforces the testing workflow through unit tests using RSpecs, integration tests, and simulated LMS environments with local canvas setups. Ensures full codebase line coverage and service interactions.

Testing Framework

- We will use the official canvas-lms app to help test functionality of our gem. This will largely be manual, automated testing can also be considered later.
- Github repo to set up a local canvas to test our ruby gem and its functionalities:
  - https://github.com/instructure/canvas-lms
  - https://github.com/instructure/canvas-lms/wiki/Quick-Start
- Additionally, we plan to create unit tests using rspec. We can use a reference implementation of the protocol in Python (https://github.com/dmitry-viskov/pylti1.3) to help with coverage.